# TensorFlow (2.20.0) for Mechanical Engineering

A self-contained Word tutorial on TensorFlow fundamentals, core layers, model building, common blocks, and beginner custom layers

Written for Mechanical Engineering students with beginner Python experience and no previous TensorFlow background

**Document features**

- Teaching sequence designed for absolute beginners
- TensorFlow 2.20.0-compatible examples that use tf.keras
- Mechanical Engineering mini-cases in regression, classification, images, and sequences
- Theory, mathematics, code, figures, citations, and references in one place
- External figures reproduced only in their complete original standalone form

| | |
|---|---|
| **Audience** | Mechanical Engineering students who know beginner Python and basic algebra, functions, vectors, and introductory calculus. |
| **Main perspective** | Engineering-first explanation of tensors, layers, blocks, models, training, inference, and custom layers. |
| **Reference date** | Mar. 28, 2026 |

*How to use this tutorial: read the sections in order the first time. Later, the layer sheets, block sheets, and model comparison tables can be used as a quick reference during coursework, lab sessions, or project work.*

### Detailed learning outline and teaching sequence

This tutorial is arranged so that each new idea depends only on ideas already explained. The aim is not only to show TensorFlow syntax, but also to teach how to think about tensors, shapes, layers, and engineering models from first principles.

1. Stage 1: orient the reader with Artificial Intelligence (AI), Machine Learning (ML), Deep Learning (DL), TensorFlow, layers, models, training, and inference.
2. Stage 2: review only the Python concepts needed to read the code examples, including variables, functions, classes, objects, methods, and keyword arguments.
3. Stage 3: introduce tensors and shape conventions for tabular data, images, and time sequences, because shape literacy is the most important survival skill in TensorFlow.
4. Stage 4: walk through one complete beginner workflow on a Mechanical Engineering-flavored regression task before discussing many individual layers.
5. Stage 5: study popular TensorFlow layers in grouped families so that similar layers are compared directly and not memorized in isolation.
6. Stage 6: step back and review the mathematics of the main layer families so the code remains connected to the underlying operations.
7. Stage 7: combine layers into common architectural blocks and then into full models using Sequential and Functional API patterns.
8. Stage 8: finish with custom layer design, Mechanical Engineering starter recipes, and a debugging checklist that students can use during projects.

#### Why this sequence works for beginners

The reader first learns vocabulary, then shape intuition, then a full training workflow, then individual building blocks, and only after that moves to more abstract architectural patterns and custom design.

**Table of contents**



## 1 Introduction to TensorFlow and its role in Mechanical Engineering

TensorFlow is an open-source software platform for machine learning. For beginners using TensorFlow 2.20.0, the most practical starting point is the high-level Keras interface inside TensorFlow. With it, a student can define layers, connect them into a model, configure training, and run evaluation and prediction using a small set of consistent steps [1], [2], [5].

In engineering language, TensorFlow is a computational framework for building models that map inputs to outputs. The inputs may be mixture properties, inspection images, or time-ordered sensor signals. The outputs may be a predicted strength value, a damage class, a future water level, or a probability distribution over several categories.

### From broad field to software tool

| Term | Meaning | Mechanical Engineering example |
|---|---|---|
| Artificial Intelligence (AI) | The broad goal of making computers perform tasks that appear intelligent. | An automated mechanical component-inspection assistant that helps interpret images. |

| | | |
|---|---|---|
| Machine Learning (ML) | A branch of AI in which models learn patterns from data. | Learning how mixture variables relate to material strength. |
| Deep Learning (DL) | A branch of ML that uses layered neural networks. | Learning crack features directly from images with a Convolutional Neural Network (CNN). |
| TensorFlow | The software framework used to build and train those models. | Writing, training, evaluating, and deploying the model in Python. |

A tensor is the basic data object in TensorFlow. A layer is one mathematical transformation that acts on tensors. A model is an organized collection of layers that maps inputs to outputs and also exposes training and evaluation tools. Training means adjusting trainable parameters to reduce a loss function on data, while inference means using the learned parameters to make predictions on new cases [2], [5], [7], [11], [12].

Mechanical engineers can use TensorFlow for many kinds of problems. Regression problems include material strength prediction and material-property estimation [19], [20]. Classification problems include material or material class prediction and machined-surface defect categorization [22]. Image-based inspection problems include crack detection and automated damage screening [21]. Sequence problems include structural health monitoring, flow-rate-coolant flow forecasting, water-level forecasting, and production throughput prediction, where the order of observations matters [14], [15], [23].

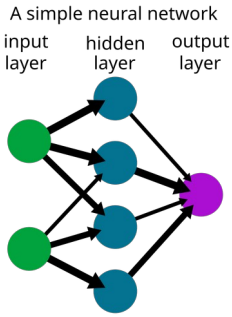


***Figure 1. A simple feed-forward neural network with an input layer, hidden layers, and an output layer. For beginners, this figure helps separate the ideas of inputs, hidden features, and outputs before looking at TensorFlow code. Source: Wiso via Wikimedia Commons, public domain [27]***

Figure 1 gives a first visual intuition for a neural network: information starts at the input, is transformed through learned hidden representations, and ends at the output. TensorFlow expresses this process by moving tensors through layers [27].

### Section summary

- TensorFlow is the software framework; machine learning and deep learning are the broader technical ideas.
- A tensor stores data, a layer transforms data, and a model combines layers into a complete input-to-output mapping.
- Mechanical Engineering use cases include tabular prediction, classification, image inspection, time-series forecasting, and monitoring.

## 2 Minimal Python mounting bases for reading TensorFlow code

This section covers only the Python ideas needed to understand the examples that follow. You do not need to be an expert programmer to start using TensorFlow effectively, but you do need to understand how Python stores values, calls functions, creates objects, and prints information for inspection.

### 2.1 Variables and functions

A variable is simply a name that refers to a value. A function groups a reusable calculation under a name. In TensorFlow tutorials, functions are often used to build reusable blocks or to compute helper values such as errors, ratios, or shape conversions.

```
cement_kg_m3 = 350.0
water_kg_m3 = 175.0

def water_cement_ratio(water, cement):
    return water / cement

ratio = water_cement_ratio(water_kg_m3, cement_kg_m3)
print("Water-cement ratio:", ratio)
```

In the example above, water and cement are function arguments. The returned value is stored in the variable ratio. This is the same basic pattern that TensorFlow uses when you pass tensors into a layer and receive a new tensor back.

### 2.2 Classes, objects, and methods

A class is a blueprint for creating objects. An object is one instance created from a class. A method is a function that belongs to an object. TensorFlow layers and models are Python objects, so it is helpful to think of them as reusable tools with stored configuration and callable behavior.

```
class InspectionRecord:
    def __init__(self, structure_name, crack_width_mm):
        self.structure_name = structure_name
        self.crack_width_mm = crack_width_mm

    def describe(self):
        print(f"{self.structure_name}: crack width = {self.crack_width_mm} mm")

record = InspectionRecord("Support mount P3", 0.42)
record.describe()
```

When you write tf.keras.layers.Dense(16), you are creating a Dense layer object. When you later call that object on an input tensor, the layer performs its stored computation.

### 2.3 Arguments and keyword arguments

A function call can use positional arguments or keyword arguments. TensorFlow code frequently uses keyword arguments because they make configuration easier to read.

```
import tensorflow as tf

layer = tf.keras.layers.Dense(
    units=16,
    activation="relu",
    use_bias=True,
)
print(layer)
```

In that example, units, activation, and use_bias are keyword arguments. This style is clearer than relying only on argument position.

### 2.4 Printing and inspecting values

Beginners should inspect objects frequently. The most useful quick checks are print(value), type(value), value.shape, and value.dtype. Shape tells you how data are organized. Data type tells you what kind of numbers are stored.

```
import tensorflow as tf

x = tf.constant([[350.0, 175.0], [400.0, 160.0]], dtype=tf.float32)
print("Type:", type(x))
print("Shape:", x.shape)
print("Data type:", x.dtype)
print(x)
```

### 2.5 Tensors at a beginner level

A tensor is a generalization of a scalar, vector, and matrix to any number of dimensions. In TensorFlow, tensors are the main carriers of data. Shapes tell you how many axes a tensor has and how large each axis is [10].

**Common beginner tensor examples**

| Tensor kind | Shape example | Mechanical Engineering interpretation |
|---|---|---|
| Scalar | () | One single number, such as one measured deflection value. |
| Vector | (8,) | Eight material-mixture features for one sample. |
| Matrix | (32, 8) | A batch of 32 tabular samples, each with 8 features. |
| 3-D tensor | (60, 8) | A 60-step sensor sequence with 8 features per step. |
| 4-D tensor | (16, 128, 128, 3) | A batch of 16 RGB inspection images. |

Shape literacy is essential. Most beginner TensorFlow errors are not mathematical errors. They are shape errors.

**Section summary**

- TensorFlow code is ordinary Python code built around objects such as layers and models.
- Keyword arguments make TensorFlow code easier to read.
- Inspecting shape and data type early prevents many beginner mistakes.

## 3 TensorFlow fundamentals

TensorFlow represents data as tensors, and almost every modeling decision can be understood as choosing a tensor shape, applying a layer, and obtaining a new tensor shape. This section builds the vocabulary needed for later code examples [2], [7], [10].

### 3.1 Tensor shapes and what dimensions mean

A shape is an ordered tuple of dimension sizes. The same numerical values can have very different meanings depending on shape. In TensorFlow, the last dimension often represents features or channels, while earlier dimensions represent batch, time, height, or width.

**Shape notation used in this tutorial**

| Symbol | Meaning | Example |
|---|---|---|
| B | Batch size | 32 samples processed together |
| F | Feature count | 8 mixture variables per sample |
| T | Sequence length or time steps | 60 sensor readings in order |
| H | Image height | 128 pixels |
| W | Image width | 128 pixels |
| C | Channel count | 3 for RGB images or 16 learned feature maps |

TensorFlow usually uses channels-last image format, written as (B, H, W, C). This is the default convention in most beginner tf.keras examples.

### 3.2 Common dimension roles

| | |
|---|---|
| **Batch dimension** | The batch dimension tells how many samples are processed together. It is not a feature of one sample. It is the number of samples in a batch. |
| **Feature dimension** | The feature dimension holds descriptive variables such as cement content, water content, age, strain, or temperature. |
| **Spatial dimensions** | Height and width tell where values lie in an image or feature map. |
| **Channels** | Channels are parallel maps. In an RGB image, channels are red, green, and blue. In a CNN feature map, channels are learned detectors. |
| **Sequence dimension** | The sequence dimension stores order in time or position. It is critical for Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and attention-based sequence models. |

### 3.3 Weights, biases, activations, and outputs

A learned layer usually performs two steps. First, it computes a weighted combination of inputs. Second, it may pass the result through an activation function. The weighted values are called weights, the optional additive terms are biases, and the transformed values are activations.

$$z = W x + b$$

$$a = \varphi(z)$$

Here, x is the input, W stores learned weights, b stores learned biases, z is the pre-activation value, φ is an activation function, and a is the output activation that is passed to the next layer.

### 3.4 Trainable and non-trainable parameters

Trainable parameters are updated during optimization. Examples include the weights and biases of Dense, Conv2D, LSTM, or GRU layers. Non-trainable parameters are values stored by the layer that are not directly updated by gradient descent. A classic example is the moving mean and moving variance inside BatchNormalization. Configuration choices such as pool size, padding mode, or the number of units are also important, but they are not parameters stored as trainable tensors.

### 3.5 The difference between a layer and a model

A layer performs one transformation on tensors. A model is a larger object that usually combines several layers into a complete mapping from inputs to outputs. In Keras, a model also provides training, evaluation, saving, and summary tools [2], [5], [7].

**Layer versus model**

| Question | Layer | Model |
|---|---|---|
| What does it do? | Transforms an input tensor into an output tensor. | Maps one or more input tensors to one or more outputs using several layers. |
| Typical scale | One operation or one small reusable block. | A complete predictor, classifier, or forecaster. |
| Can it hold weights? | Yes, depending on layer type. | Yes, through the layers it contains. |
| Can it train on data? | Not by itself in the usual beginner workflow. | Yes, via compile, fit, evaluate, and predict. |

### 3.6 Three beginner model-building styles to know

| | |
|---|---|
| **Sequential style** | Best for a straight stack of layers, where one layer feeds directly into the next. This is the easiest starting point. |
| **Functional API** | Best when the graph branches, merges, has several inputs or outputs, or uses skip connections. |
| **Custom layer creation** | Best when the computation itself is not available as a standard built-in layer and you want to define a new reusable layer class. |

### 3.7 A quick shape-inspection habit

```
import tensorflow as tf

x_tabular = tf.random.normal((32, 8))
x_image = tf.random.normal((16, 128, 128, 3))
x_sequence = tf.random.normal((10, 60, 5))

print("Tabular:", x_tabular.shape)
print("Image:", x_image.shape)
print("Sequence:", x_sequence.shape)
print("Dynamic image shape:", tf.shape(x_image))
```

The expression x.shape gives the static shape known from the Python object, while tf.shape(x) returns a TensorFlow tensor containing the dynamic shape values. Both are useful in debugging [10].

**Section summary**

- TensorFlow models are easiest to understand by following tensor shapes from input to output.
- Weights and biases are trainable. Batch normalization statistics are a common non-trainable example.
- The three beginner design styles are Sequential, Functional API, and custom layer creation.

## 4 A first end-to-end TensorFlow workflow

Before studying many individual layers, it helps to see the full workflow once. The example below uses synthetic data shaped like a material-mixture regression problem. The same workflow transfers directly to real engineering datasets.

The mechanical material strength dataset in the UCI Machine Learning Repository is a classic beginner-friendly example with 8 input variables and 1 target value. It is based on the work of Yeh on high-performance material strength prediction [19], [20].

### The standard beginner workflow

| Step | What happens |
|---|---|
| 1. Prepare tensors | Organize inputs X and targets y with the correct shape and data type. |
| 2. Define the model | Choose layers and connect them into an input-to-output mapping. |
| 3. Compile | Choose optimizer, loss, and evaluation metrics. |
| 4. Fit | Run training over the data for several epochs. |
| 5. Evaluate | Check model performance on validation or test data. |
| 6. Predict | Run inference on new samples. |

```
import numpy as np
import tensorflow as tf

tf.random.set_seed(42)
np.random.seed(42)

# Synthetic tabular data shaped like a material-mixture problem.
# 8 input features could represent cement, slag, fly ash, water,
# superplasticizer, coarse aggregate, fine aggregate, and age.
X = np.random.rand(200, 8).astype("float32")
y = (20 + 15 * X[:, 0] - 8 * X[:, 3] + 5 * X[:, 7]).astype("float32")

model = tf.keras.Sequential(
    [
        tf.keras.Input(shape=(8,), name="mix_features"),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(1, name="strength_mpa"),
    ],
    name="concrete_strength_model",
)

model.compile(
    optimizer="adam",
    loss="mse",
```

```
    metrics=["mae"],
)

history = model.fit(
    X,
    y,
    epochs=5,
    batch_size=32,
    verbose=0,
)

pred = model.predict(X[:3], verbose=0)
print("Prediction shape:", pred.shape)
print("Predicted strengths (MPa):", pred[:, 0])
```

Line by line, the workflow is simple. The input tensor X has shape (200, 8), meaning 200 samples and 8 features. The target vector y contains one strength value per sample. The model starts with an explicit Input layer, then two hidden Dense layers with ReLU activation, and finally one output unit because this is a regression problem. The compile step chooses Adam as the optimizer, mean squared error (MSE) as the loss, and mean absolute error (MAE) as a readable metric. The fit step performs training, and predict performs inference on new samples.

For real course or project work, synthetic X and y would be replaced with measured data. The structural pattern of the code would remain the same.

| | |
|---|---|
| **Loss** | A numerical measure of how wrong the predictions are. Training tries to reduce it. |
| **Optimizer** | The algorithm that updates trainable parameters using gradients. Adam is a common beginner default. |
| **Metric** | A human-readable performance score such as mean absolute error or accuracy. |
| **Epoch** | One full pass through the training data. |
| **Inference** | Using the trained model to produce predictions on new inputs. |

### Section summary

- The end-to-end workflow is prepare data, define model, compile, fit, evaluate, and predict.
- A simple Dense network is a good first baseline for many tabular Mechanical Engineering problems.
- Once you understand this workflow, individual layers become easier to place in context.

## 5 Popular TensorFlow layers

This section uses a consistent format for each layer: intuition, purpose, input and output meaning, trainable state, simple mathematics, a short TensorFlow 2.20.0-compatible example, and a Mechanical Engineering interpretation. Read the layer sheets slowly the first time. Later, they can be used as a lookup reference.

### 5.1 Dense, activation, normalization, and regularization layers

These layers dominate beginner tabular models and also appear inside image and sequence models. Dense layers learn new feature combinations. Activation layers introduce nonlinearity. BatchNormalization and Dropout are training-support layers [24], [25], [26].

#### 5.1.1 Dense

| | |
|---|---|
| **Plain-language idea** | A Dense layer lets every output feature look at every input feature. It is the standard fully connected layer for tabular data and for the final decision part of many networks. |
| **Purpose and where used** | Used in regression and classification, especially for tabular problems such as material strength prediction, material-property regression, or component condition scoring [7], [11], [12]. |
| **Typical input and output** | Input is usually (batch, features). Output is (batch, units). Each sample becomes a new learned feature vector. |
| **Trainable and non-trainable quantities** | Trainable: weight matrix W and bias vector b. Non-trainable: none, unless the layer is wrapped inside another layer with fixed behavior. |
| **Mechanical Engineering interpretation** | A Dense layer can learn how cement content, water content, aggregate content, and age combine to influence compressive strength. |

$$y = \varphi(x\,W + b)$$

```
import tensorflow as tf

# 4 material-mixture samples, each with 8 numeric features.
x = tf.random.uniform((4, 8), dtype=tf.float32)

dense = tf.keras.layers.Dense(units=16, activation="relu")
y = dense(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
print("Trainable weights:", [w.shape for w in dense.trainable_weights])
```

Dense layers are often the default first choice for spreadsheet-like data.

#### 5.1.2 ReLU

| | |
|---|---|
| **Plain-language idea** | ReLU stands for Rectified Linear Unit. It keeps positive values and sets negative values to zero. |
| **Purpose and where** | It is a simple default activation for hidden layers because it is easy to optimize and |

| | |
|---|---|
| **used** | works well in many practical models [11], [24]. |
| **Typical input and output** | Input and output shapes are identical. ReLU acts element by element. |
| **Trainable and non-trainable quantities** | No trainable parameters. No stored non-trainable state. |
| **Mechanical Engineering interpretation** | Useful in hidden layers that learn nonlinear relationships among material properties, image features, or sensor readings. |

$$ReLU(x) = max(0, x)$$

```
import tensorflow as tf

x = tf.constant([[-2.0, -0.5, 0.0, 1.5, 3.0]])
relu = tf.keras.layers.ReLU()
y = relu(x)

print("Input:", x.numpy())
print("Output:", y.numpy())
```

ReLU is usually used between learned layers such as Dense or Conv2D.

### 5.1.3 LeakyReLU

| | |
|---|---|
| **Plain-language idea** | LeakyReLU is like ReLU, but it keeps a small negative slope instead of becoming exactly zero for negative inputs. |
| **Purpose and where used** | Used when you want ReLU-like simplicity but less risk that many activations become permanently inactive. |
| **Typical input and output** | Input and output shapes are identical. The layer acts element by element. |
| **Trainable and non-trainable quantities** | Usually no trainable parameters. The negative slope is a fixed configuration value. |
| **Mechanical Engineering interpretation** | Can help when a model should still react to weak negative evidence, such as small deviations in vibration or hydraulic signals. |

$$LeakyReLU(x) = max(\alpha\, x, x), \text{ with a small } \alpha > 0$$

```
import tensorflow as tf

x = tf.constant([[-2.0, -0.5, 0.0, 1.5, 3.0]])
act = tf.keras.layers.LeakyReLU(negative_slope=0.1)
y = act(x)
```

```
print("Input:", x.numpy())
print("Output:", y.numpy())
```

LeakyReLU is a small change, but sometimes it improves stability in deeper networks.

### ReLU versus LeakyReLU

| Aspect | ReLU | LeakyReLU |
|---|---|---|
| Negative input behavior | Outputs exactly zero. | Keeps a small negative slope. |
| Simplicity | Very simple and widely used. | Almost as simple. |
| Typical use | Default hidden activation. | Alternative when zeroing too many activations is a concern. |
| Output shape | Same as input. | Same as input. |

### 5.1.4 Softmax

| | |
|---|---|
| **Plain-language idea** | Softmax turns raw class scores into positive values that sum to 1, so they can be interpreted as class probabilities. |
| **Purpose and where used** | Used at the output of multi-class classifiers, for example machined-surface defect type classification or material category classification. |
| **Typical input and output** | If input is (batch, classes), output is also (batch, classes). Each row sums to 1. |
| **Trainable and non-trainable quantities** | No trainable parameters. No stored non-trainable state. |
| **Mechanical Engineering interpretation** | If a surface image can belong to one of several distress classes, Softmax gives a probability for each class. |

$$Softmax(z\_i) = exp(z\_i) / \Sigma\_j\ exp(z\_j)$$

```
import tensorflow as tf

logits = tf.constant([[1.2, 0.3, -0.7]])
softmax = tf.keras.layers.Softmax()
probs = softmax(logits)

print("Probabilities:", probs.numpy())
print("Row sum:", tf.reduce_sum(probs, axis=-1).numpy())
```

Softmax belongs at the final layer of a multi-class classifier, not inside a hidden layer.

### 5.1.5 Dropout

| | |
|---|---|
| **Plain-language idea** | During training, Dropout randomly turns off some activations. This makes the |

| | |
|---|---|
| | model rely less on any one path. |
| **Purpose and where used** | Used as regularization to reduce overfitting, especially when training data are limited [26]. |
| **Typical input and output** | Input and output shapes are identical. The layer acts element by element during training. |
| **Trainable and non-trainable quantities** | No trainable parameters. No stored non-trainable state. |
| **Mechanical Engineering interpretation** | Helpful when a distress-image dataset or material dataset is small, which is common in engineering projects. |

$$x' = m \odot x, \text{ where each mask entry } m \text{ is randomly } 0 \text{ or } 1 \text{ during training}$$

```
import tensorflow as tf

tf.random.set_seed(42)
x = tf.ones((2, 6))
drop = tf.keras.layers.Dropout(rate=0.5)

y_train = drop(x, training=True)
y_infer = drop(x, training=False)

print("Training output:\n", y_train.numpy())
print("Inference output:\n", y_infer.numpy())
```

Dropout changes behavior between training and inference. That difference matters.

### 5.1.6 BatchNormalization

| | |
|---|---|
| **Plain-language idea** | BatchNormalization standardizes intermediate activations and then lets the network rescale them. |
| **Purpose and where used** | It often stabilizes and speeds up training in deeper networks [25]. |
| **Typical input and output** | Input and output shapes are identical. The layer normalizes along the feature or channel dimension, depending on context. |
| **Trainable and non-trainable quantities** | Trainable: γ (scale) and β (shift). Non-trainable: moving mean and moving variance used at inference time. |
| **Mechanical Engineering interpretation** | Useful in deeper image models for crack detection or distress classification where training can otherwise become unstable. |

$$\hat{x} = (x - \mu) / sqrt(\sigma^2 + \varepsilon), \quad y = \gamma \hat{x} + \beta$$

```
import tensorflow as tf

x = tf.random.normal((4, 8))
bn = tf.keras.layers.BatchNormalization()
y = bn(x, training=True)

print("Output shape:", y.shape)
print("Trainable weights:", [w.shape for w in bn.trainable_weights])
print("Non-trainable weights:", [w.shape for w in bn.non_trainable_weights])
```

BatchNormalization also behaves differently during training and inference.

### 5.2 Image, spatial, and tensor-shape layers

These layers are central to image models. They preserve or deliberately manipulate spatial layout, channel depth, and feature-map size. For inspection and crack analysis tasks, these are often the most important layers to understand [8], [11], [12].

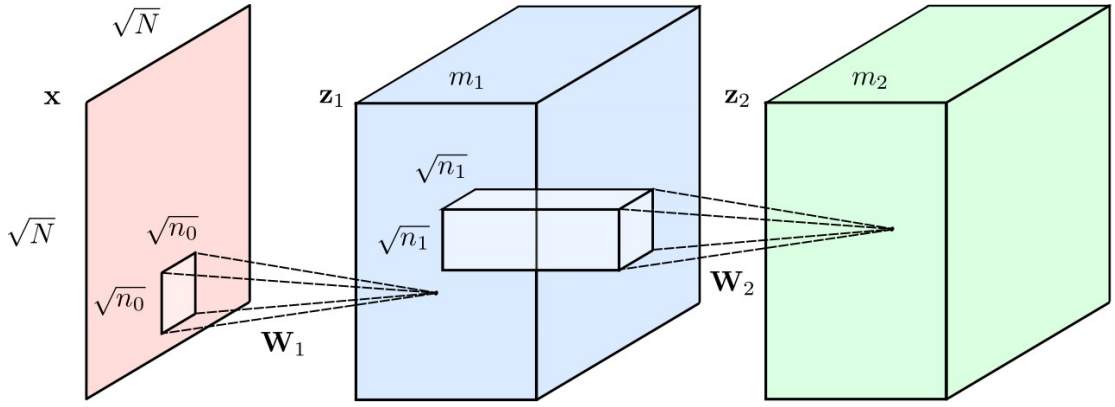


***Figure 2. Convolutional layers can change spatial resolution and channel depth while preserving local spatial structure. The figure shows how filters operate on local regions and produce new feature maps. Source: Renanar2 via Wikimedia Commons, CC BY-SA 4.0 [28]***

Figure 2 is useful because it shows two ideas at once: convolution works locally in space, and deeper layers often trade larger channel depth for smaller spatial size [28].

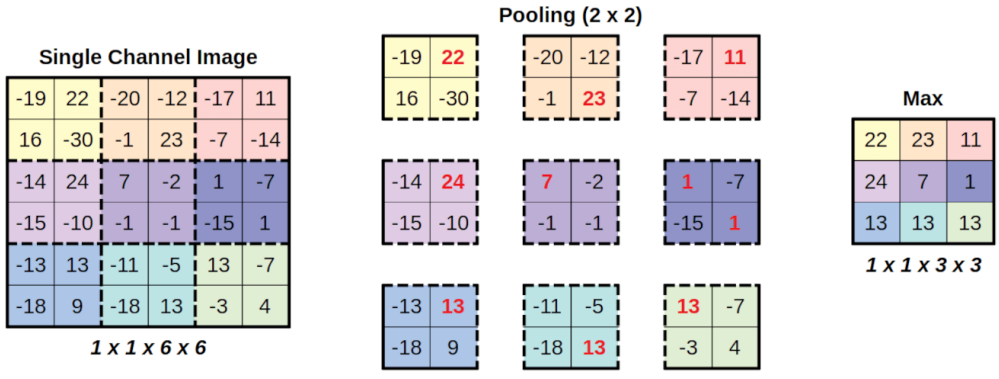


***Figure 3. Example of max pooling on feature maps. Pooling keeps strong responses while reducing spatial resolution. Source: Daniel Voigt Godoy via Wikimedia Commons, CC BY 4.0 [29]***

Figure 3 highlights the fixed-rule nature of pooling. Unlike convolution, the pooling step does not learn filters [29].

### 5.2.1 Conv2D

| | |
|---|---|
| **Plain-language idea** | Conv2D slides small learnable filters across an image. Each filter looks for a local pattern such as an edge, texture, or crack-like shape. |
| **Purpose and where used** | Core layer for image analysis, feature extraction, and computer vision [8], [11], [12]. |
| **Typical input and output** | Input is usually (batch, height, width, channels). Output is (batch, new_height, new_width, filters). |
| **Trainable and non-trainable quantities** | Trainable: filter weights and usually one bias per filter. Non-trainable: none. |
| **Mechanical Engineering interpretation** | Used for machined-surface defect images, crack images, drone inspection imagery, and damage-region analysis. |

$$y[i, j, k] = \Sigma\_u \Sigma\_v \Sigma\_c W[u, v, c, k] \cdot x[i+u, j+v, c] + b\_k$$

```
import tensorflow as tf

# 2 RGB surface images, each 128 x 128.
x = tf.random.normal((2, 128, 128, 3))

conv = tf.keras.layers.Conv2D(
    filters=16,
    kernel_size=3,
    strides=1,
    padding="same",
    activation="relu",
)
y = conv(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

Convolution reuses the same filter weights across the image. That weight sharing is the key difference from Dense layers.

### Dense versus Conv2D

| Aspect | Dense | Conv2D |
|---|---|---|
| Connection pattern | Each output sees every input feature. | Each output sees only a local spatial neighborhood. |
| Weight sharing | No weight sharing across input position. | The same filter is reused across spatial positions. |
| Typical input | (batch, features) | (batch, height, width, channels) |
| Strength | Strong default for tabular data. | Strong default for images and feature |

| | | |
|---|---|---|
| | | maps. |
| Mechanical Engineering example | Material strength regression. | Crack or machined surface-image inspection. |

### 5.2.2 ZeroPadding2D

| | |
|---|---|
| **Plain-language idea** | ZeroPadding2D adds zeros around the border of an image or feature map. |
| **Purpose and where used** | Used when you want to preserve edge information, control output size, or align feature-map shapes before later operations. |
| **Typical input and output** | If input is (batch, H, W, C), output becomes (batch, H+padding, W+padding, C). |
| **Trainable and non-trainable quantities** | No trainable parameters. No stored non-trainable state. |
| **Mechanical Engineering interpretation** | Helps a crack detector keep information near image boundaries, where damage may continue out of frame. |

*The layer only changes shape. It does not learn weights or alter inner pixel values.*

```
import tensorflow as tf

x = tf.random.normal((1, 5, 5, 1))
pad = tf.keras.layers.ZeroPadding2D(padding=1)
y = pad(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

Padding is a shape-control operation, not a learned feature extractor.

### 5.2.3 MaxPooling2D

| | |
|---|---|
| **Plain-language idea** | MaxPooling2D keeps the strongest value in each small window. It reduces spatial size while preserving strong responses. |
| **Purpose and where used** | Used to downsample feature maps, reduce memory use, and build some translation robustness. |
| **Typical input and output** | Input is (batch, H, W, C). Output is (batch, smaller_H, smaller_W, C). Channels stay the same. |
| **Trainable and non-trainable quantities** | No trainable parameters. No stored non-trainable state. |
| **Mechanical** | Can help a CNN keep strong crack or spalling responses while reducing image |

| | |
|---|---|
| **Engineering interpretation** | resolution. |

$$y[i, j, c] = max\ over\ the\ pooling\ window\ taken\ from\ x$$

```
import tensorflow as tf

x = tf.random.normal((1, 64, 64, 16))
pool = tf.keras.layers.MaxPooling2D(pool_size=2, strides=2)
y = pool(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

Pooling has no weights. It is a fixed rule.

### 5.2.4 GlobalAveragePooling2D

| | |
|---|---|
| **Plain-language idea** | This layer averages each feature map into one number. One channel becomes one summary value. |
| **Purpose and where used** | Common near the end of image classifiers because it is lighter than Flatten and reduces parameter count. |
| **Typical input and output** | Input is (batch, H, W, C). Output is (batch, C). |
| **Trainable and non-trainable quantities** | No trainable parameters. No stored non-trainable state. |
| **Mechanical Engineering interpretation** | A crack-classification model can compress many local crack features into one vector before the final classifier. |

$$y[k] = (1 / (HW)) \Sigma\_i \Sigma\_j x[i, j, k]$$

```
import tensorflow as tf

x = tf.random.normal((2, 16, 16, 32))
gap = tf.keras.layers.GlobalAveragePooling2D()
y = gap(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

Global average pooling is often a cleaner finish for CNN classifiers than Flatten plus a large Dense layer.

### 5.2.5 Flatten

| | |
|---|---|
| **Plain-language idea** | Flatten turns many non-batch dimensions into one long vector. |

| | |
|---|---|
| **Purpose and where used** | Used when a later Dense layer expects a vector instead of an image-like tensor. |
| **Typical input and output** | Input may be (batch, H, W, C). Output becomes (batch, H·W·C). |
| **Trainable and non-trainable quantities** | No trainable parameters. No stored non-trainable state. |
| **Mechanical Engineering interpretation** | A small image classifier for machined surface patches may use Flatten before the final Dense classifier. |

*The layer only changes shape. It does not change the numerical values.*

```
import tensorflow as tf

x = tf.random.normal((2, 16, 16, 8))
flat = tf.keras.layers.Flatten()
y = flat(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

Flatten can create very large vectors. GlobalAveragePooling2D is often a lighter alternative.

### 5.2.6 Reshape

| | |
|---|---|
| **Plain-language idea** | Reshape changes how TensorFlow interprets the arrangement of values without changing the values themselves. |
| **Purpose and where used** | Used to convert vectors into sequences, sequences into grids, or one convenient shape into another. |
| **Typical input and output** | The total number of elements must stay the same. For example, (batch, 60) can become (batch, 12, 5). |
| **Trainable and non-trainable quantities** | No trainable parameters. No stored non-trainable state. |
| **Mechanical Engineering interpretation** | Helpful when sensor readings arrive as a flat vector but should be interpreted as a time sequence. |

*Total element count before and after reshaping must match.*

```
import tensorflow as tf

x = tf.random.normal((4, 60))
reshape = tf.keras.layers.Reshape((12, 5))
y = reshape(x)
```

```
print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

Reshape is useful, but only when the new organization still represents the data correctly.

### 5.2.7 UpSampling2D

| | |
|---|---|
| **Plain-language idea** | UpSampling2D enlarges feature maps by repeating rows and columns. |
| **Purpose and where used** | Used in decoders, segmentation networks, and image-to-image tasks when a larger spatial grid is needed. |
| **Typical input and output** | Input is (batch, H, W, C). Output is (batch, larger_H, larger_W, C). |
| **Trainable and non-trainable quantities** | No trainable parameters. No stored non-trainable state. |
| **Mechanical Engineering interpretation** | Useful when a model should recover a damage-location heat map from compressed image features. |

*This layer copies values into a larger grid. It is not learned.*

```
import tensorflow as tf

x = tf.random.normal((1, 32, 32, 16))
up = tf.keras.layers.UpSampling2D(size=2)
y = up(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

A common design is UpSampling2D followed by Conv2D for smoother learned refinement.

### 5.2.8 Conv2DTranspose

| | |
|---|---|
| **Plain-language idea** | Conv2DTranspose performs learned upsampling. It can increase image size while also learning what should appear in the new pixels. |
| **Purpose and where used** | Used in decoders, segmentation networks, and convolutional autoencoders. |
| **Typical input and output** | Input is (batch, H, W, C_in). Output is a larger spatial tensor such as (batch, 2H, 2W, filters). |
| **Trainable and non-trainable quantities** | Trainable: filter weights and biases. Non-trainable: none. |
| **Mechanical Engineering interpretation** | Useful for reconstructing damage-location maps or refining low-resolution image features into a denser output. |

*It is a learned spatial upsampling operator. In practice, it is often used as the decoder-side counterpart of Conv2D.*

```
import tensorflow as tf

x = tf.random.normal((1, 32, 32, 16))
deconv = tf.keras.layers.Conv2DTranspose(
    filters=8,
    kernel_size=3,
    strides=2,
    padding="same",
)
y = deconv(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

Beginners should know that transposed convolution is learned, while UpSampling2D is fixed.

### 5.2.9 Add

| | |
|---|---|
| **Plain-language idea** | Add combines tensors by elementwise addition. |
| **Purpose and where used** | Used in residual connections and any situation where two same-shaped feature sets should be merged without increasing dimension. |
| **Typical input and output** | All input tensors must have the same shape. The output has that same shape. |
| **Trainable and non-trainable quantities** | No trainable parameters. No stored non-trainable state. |
| **Mechanical Engineering interpretation** | Useful in deep crack-detection networks, where skip paths help preserve useful earlier image information. |

$$y = x_1 + x_2 + ... + x\_n$$

```
import tensorflow as tf

a = tf.random.normal((2, 32))
b = tf.random.normal((2, 32))
add = tf.keras.layers.Add()
y = add([a, b])

print("Output shape:", y.shape)
```

Add does not change the number of channels or features.

### 5.2.10 Concatenate

| | |
|---|---|
| **Plain-language idea** | Concatenate joins tensors along a chosen axis. |
| **Purpose and where** | Used to combine feature sets from different branches, scales, or data sources. |

| | |
|---|---|
| **used** | |
| **Typical input and output** | All tensors must match on non-concatenated axes. The concatenation axis becomes longer. |
| **Trainable and non-trainable quantities** | No trainable parameters. No stored non-trainable state. |
| **Mechanical Engineering interpretation** | A useful fusion operation when combining image features with metadata such as production throughput level, mechanical component age, or material type. |

*If axis = -1, features are stacked side by side along the last dimension.*

```
import tensorflow as tf

image_features = tf.random.normal((2, 64))
metadata_features = tf.random.normal((2, 8))
concat = tf.keras.layers.Concatenate(axis=-1)
y = concat([image_features, metadata_features])

print("Output shape:", y.shape)
```

Concatenate increases dimension, while Add preserves it.

### Add versus Concatenate

| Aspect | Add | Concatenate |
|---|---|---|
| Shape requirement | Inputs must match exactly. | Inputs must match on all non-concatenated axes. |
| Output dimension | Preserved. | Increases along the chosen axis. |
| Typical role | Residual connection. | Feature fusion from different branches. |
| Mechanical Engineering example | Deep crack classifier skip path. | Image features plus mechanical component metadata. |

### 5.3 Sequence, categorical, and attention-related layers

Sequence layers are used when order matters, such as for sensor signals, flow-rate histories, water-level sequences, or production observations. Attention adds a learned focus mechanism over ordered information [9], [14], [15], [18].

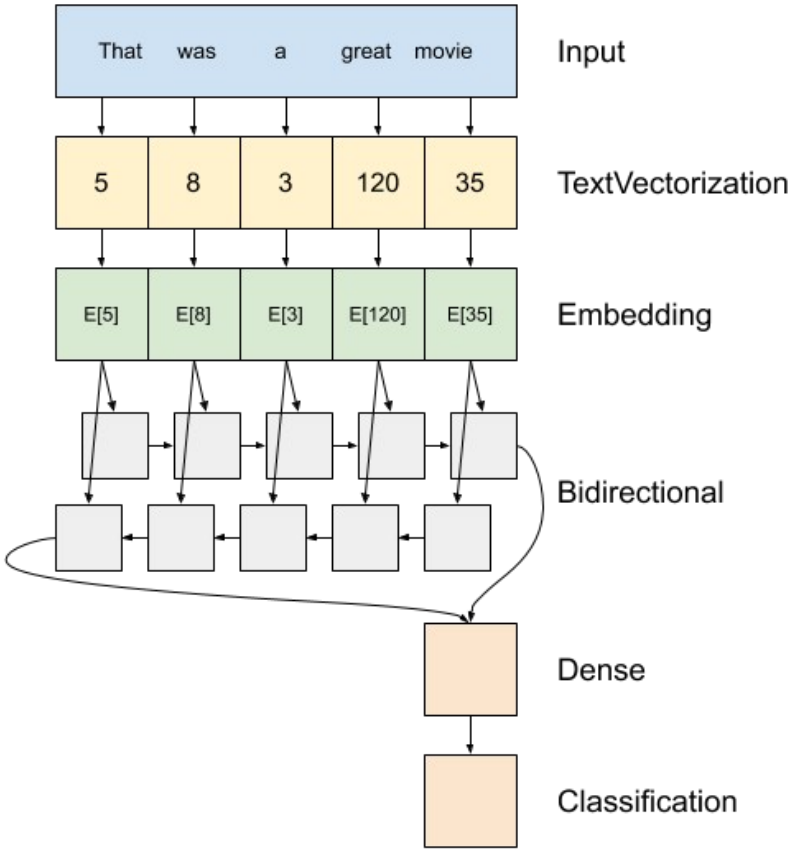


***Figure 4. Embedding vectors followed by a bidirectional recurrent structure. Although drawn for text, the same tensor idea applies to any ordered integer-coded sequence or windowed sequence data. Source: TensorFlow Tutorials [33]***

Figure 4 is from a TensorFlow tutorial on text classification, but the diagram is still useful for engineering students because it clarifies how embeddings and bidirectional recurrence change tensor flow across a sequence [33].

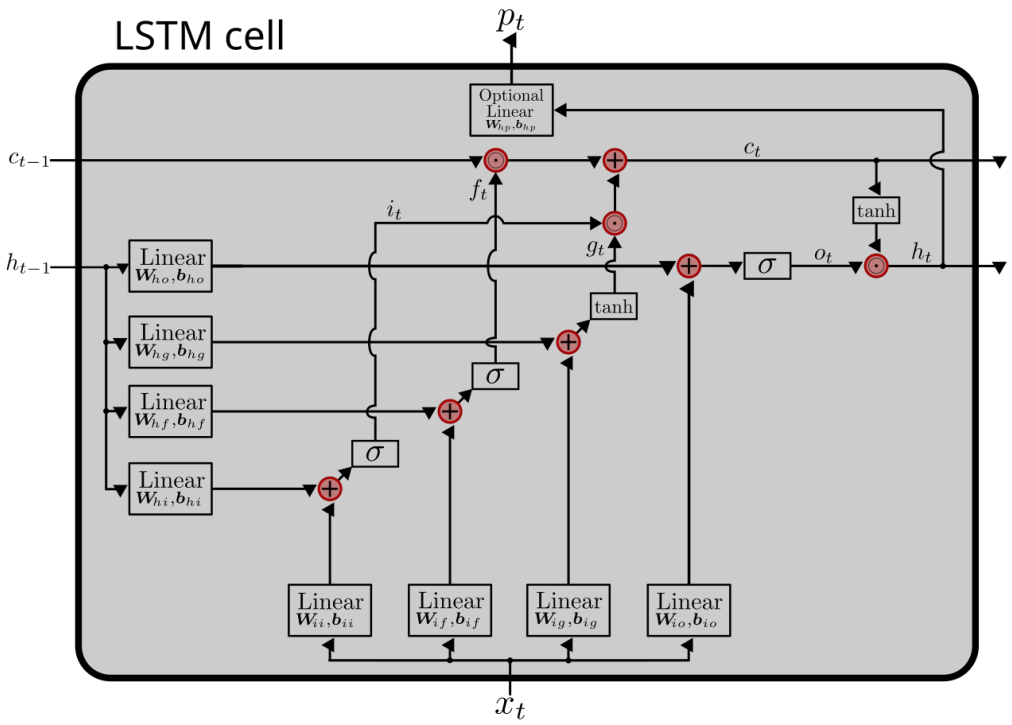


*Figure 5. An LSTM cell showing gated information flow and a memory path across time. Source: See-Ming Lee via Wikimedia Commons, CC0 1.0 [31]*

Figure 5 helps explain why LSTM is more expressive than a plain recurrent layer: the cell state provides a separate memory path controlled by gates [31].

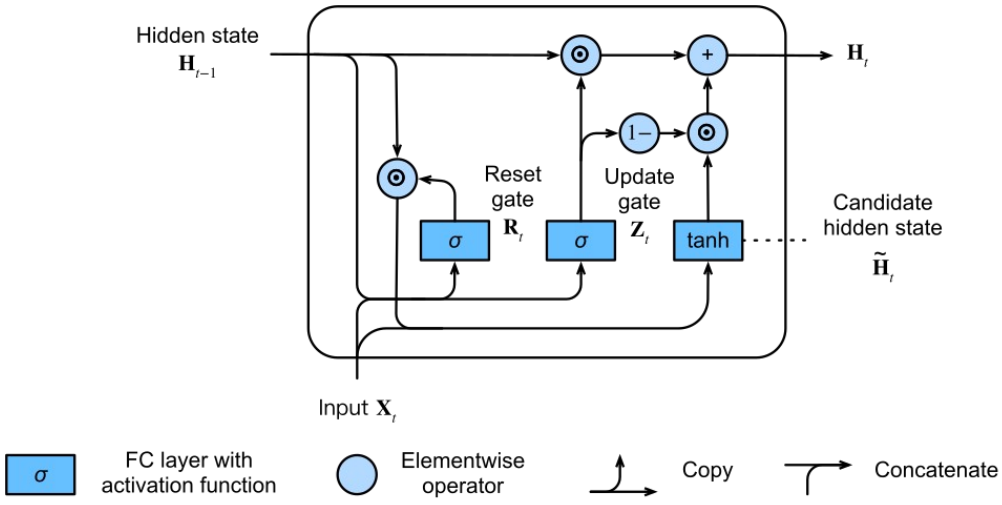


*Figure 6. A GRU cell. It is simpler than an LSTM cell and uses a smaller gating structure. Source: Wikimedia Commons contributors [32]*

Figure 6 shows why GRU is often described as a simpler gated recurrent design: it uses fewer internal moving parts than LSTM while still controlling information flow over time [32].

### 5.3.1 Embedding

| | |
|---|---|
| **Plain-language idea** | Embedding turns integer identifiers into learned dense vectors. |
| **Purpose and where** | Useful when inputs are discrete categories such as material IDs, production line |

| | |
|---|---|
| **used** | class IDs, station IDs, or inspection-code tokens. |
| **Typical input and output** | Input is usually (batch, sequence_length) of integer indices. Output is (batch, sequence_length, embedding_dim). |
| **Trainable and non-trainable quantities** | Trainable: embedding matrix. Non-trainable: none. |
| **Mechanical Engineering interpretation** | If different production line classes are encoded as integers, an embedding can learn a compact vector for each class instead of using sparse one-hot encoding. |

*Each integer index selects one learned row from an embedding matrix E.*

```
import tensorflow as tf

road_type_ids = tf.constant([[1, 3, 2, 4], [2, 1, 5, 0]])
embedding = tf.keras.layers.Embedding(input_dim=10, output_dim=6)
y = embedding(road_type_ids)

print("Input shape:", road_type_ids.shape)
print("Output shape:", y.shape)
```

Embedding inputs must be integer indices, not ordinary floating-point measurements.

### 5.3.2 LSTM

| | |
|---|---|
| **Plain-language idea** | LSTM stands for Long Short-Term Memory. It is a recurrent layer designed to keep and update memory across time steps. |
| **Purpose and where used** | Used for ordered data such as structural health monitoring signals, flow-rate-coolant flow sequences, water levels, or machine-cycle counts [14]. |
| **Typical input and output** | Input is usually (batch, time_steps, features). Output is (batch, units) or (batch, time_steps, units) if return_sequences=True. |
| **Trainable and non-trainable quantities** | Trainable: gate weights and biases. Non-trainable: none, unless wrapped by another layer with stored state. |
| **Mechanical Engineering interpretation** | An LSTM can learn how earlier sensor readings influence later structural or hydraulic behavior. |

*LSTM uses input, forget, and output gates to control a memory cell c_t and a hidden state h_t.*

```
import tensorflow as tf

# 24 time steps, 5 sensor features.
x = tf.random.normal((3, 24, 5))
lstm = tf.keras.layers.LSTM(32, return_sequences=False)
y = lstm(x)
```

```
print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

LSTM is usually the safer first recurrent choice when long-term dependencies matter.

### 5.3.3 GRU

| | |
|---|---|
| **Plain-language idea** | GRU stands for Gated Recurrent Unit. It is a recurrent layer similar to LSTM but with a simpler internal design. |
| **Purpose and where used** | Used for sequence modeling when you want recurrent behavior with fewer parameters than a typical LSTM [15]. |
| **Typical input and output** | Input and output shape rules are the same basic pattern as LSTM. |
| **Trainable and non-trainable quantities** | Trainable: gate weights and biases. Non-trainable: none. |
| **Mechanical Engineering interpretation** | GRU can be a strong starting point for short- to medium-length sensor or production throughput sequences when computation or data are limited. |

*GRU uses update and reset gates to mix previous memory with a new candidate state.*

```
import tensorflow as tf

x = tf.random.normal((3, 24, 5))
gru = tf.keras.layers.GRU(32, return_sequences=False)
y = gru(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

GRU often trains a little faster than LSTM, although the best choice depends on the task.

### LSTM versus GRU

| Aspect | LSTM | GRU |
|---|---|---|
| Memory structure | Separate cell state and hidden state. | Single hidden-state style memory. |
| Parameter count | Usually larger. | Usually smaller. |
| Typical use | When long-term dependencies are important. | When a lighter recurrent layer is preferred. |
| Mechanical Engineering example | Long water-level history forecasting. | Compact sensor-sequence classification. |

### 5.3.4 Bidirectional

| | |
|---|---|
| **Plain-language idea** | Bidirectional wraps a recurrent layer so one copy reads the sequence forward and another reads it backward. |
| **Purpose and where used** | Used when the full sequence is available before prediction and both past and later context are useful. |
| **Typical input and output** | Input shape is the same as the wrapped recurrent layer. Output often doubles the feature size because forward and backward outputs are concatenated. |
| **Trainable and non-trainable quantities** | Trainable quantities come from the wrapped recurrent layers. Bidirectional itself only controls how they are combined. |
| **Mechanical Engineering interpretation** | Useful for classifying a fixed vibration window or a full inspection sequence, but not the best default for strict future forecasting where later time steps are unavailable. |

*The output typically combines h_forward and h_backward, often by concatenation.*

```
import tensorflow as tf

x = tf.random.normal((3, 24, 5))
bi = tf.keras.layers.Bidirectional(
    tf.keras.layers.GRU(16, return_sequences=False)
)
y = bi(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

Bidirectional processing is powerful for classification or tagging, but you should think carefully before using it for forecasting.

### 5.3.5 Attention

| | |
|---|---|
| **Plain-language idea** | Attention lets one tensor focus on the most relevant parts of another tensor. |
| **Purpose and where used** | Useful when the model should learn which time steps, positions, or features matter most for a decision [9], [18]. |
| **Typical input and output** | For the basic Attention layer, inputs are usually [query, value] or [query, value, key]. If query is (batch, Tq, d) and value is (batch, Tv, d), output is (batch, Tq, d). |
| **Trainable and non-trainable quantities** | The basic Attention layer itself may have no trainable state unless scaling is enabled. The trainable behavior often comes from surrounding layers that produce query, key, and value tensors. |
| **Mechanical Engineering interpretation** | An attention layer can learn which past flow-rate steps are most important for predicting future water level, or which sensor time steps matter most for damage classification. |

$$context = softmax(Q\ K^T)\ V \quad \text{for basic dot-product attention; a common scaled form uses } softmax(Q\ K^T / \sqrt{d})$$

```
import tensorflow as tf

query = tf.random.normal((2, 10, 32))
value = tf.random.normal((2, 20, 32))
attn = tf.keras.layers.Attention()
context, scores = attn(
    [query, value],
    return_attention_scores=True,
)

print("Context shape:", context.shape)
print("Score shape:", scores.shape)
```

Attention does not replace basic recurrent or convolutional thinking. It adds a learned focus mechanism.

### Section summary

- Dense layers dominate tabular learning, Conv2D dominates image feature extraction, and recurrent plus attention layers are designed for ordered data.
- Several layers, such as Flatten, Reshape, Add, Concatenate, pooling, padding, and upsampling, have no trainable parameters but still change the computation in important ways.
- The key beginner skill is to interpret input and output shapes correctly for every layer.

## 6 Theory and mathematics by layer family

The goal of this section is not to turn the tutorial into a proof-heavy textbook. Instead, it gives just enough mathematics to explain what each family of layers is computing and why that computation is useful in engineering tasks.

### 6.1 Dense layers

Dense layers compute weighted combinations of all input features. For one output unit j, the basic formula is z_j = Σ_i w_ij x_i + b_j. An activation function may then produce a_j = φ(z_j). Dense layers are powerful for tabular Mechanical Engineering data because each output can use all available input variables at once [11], [12].

$$z_j = \Sigma_i w_{ij} x_i + b_j$$

$$a_j = \varphi(z_j)$$

For beginners, the most important idea is not the formula alone. It is the connection between the formula and the data type: tabular data suit Dense layers, images suit convolutional layers, and ordered signals suit recurrent or attention-based layers.

### 6.2 Convolutional layers

A convolutional layer applies a small learnable filter over local neighborhoods. Instead of connecting every output to every input, it uses a local receptive field and reuses the same filter weights across the image. This creates translation-aware feature extraction and a far smaller parameter count than a Dense layer on the raw pixels.

$$y[i, j, k] = \Sigma_u \Sigma_v \Sigma_c W[u, v, c, k] x[i+u, j+v, c] + b_k$$

For beginners, the most important idea is not the formula alone. It is the connection between the formula and the data type: tabular data suit Dense layers, images suit convolutional layers, and ordered signals suit recurrent or attention-based layers.

### 6.3 Pooling layers

Pooling layers reduce spatial resolution by applying a fixed rule within each window. Max pooling takes the largest value in each window. Average pooling takes the mean. Pooling is helpful because it shrinks feature maps and keeps only summary information needed by later layers.

$$y[i, j, c] = max\ over\ the\ pooling\ window\ taken\ from\ x$$

For beginners, the most important idea is not the formula alone. It is the connection between the formula and the data type: tabular data suit Dense layers, images suit convolutional layers, and ordered signals suit recurrent or attention-based layers.

### 6.4 Activation layers

Activation layers introduce nonlinearity. Without them, many stacked learned layers would still behave like one linear transformation. ReLU uses max(0, x), LeakyReLU uses max(αx, x), and Softmax turns a vector of scores into a probability distribution across classes [24].

$$ReLU(x) = max(0, x)$$

$$Softmax(z_i) = exp(z_i) / \Sigma_j exp(z_j)$$

For beginners, the most important idea is not the formula alone. It is the connection between the formula and the data type: tabular data suit Dense layers, images suit convolutional layers, and ordered signals suit recurrent or attention-based layers.

### 6.5 Recurrent layers

Recurrent layers process one time step after another while carrying state forward. LSTM and GRU improve plain recurrence by learning gates that control what information should be remembered, updated, or discarded. This is especially useful when later outputs depend on earlier context.

$$f_t = \sigma(W_f [x_t, h_{t-1}] + b_f), \quad i_t = \sigma(W_i [x_t, h_{t-1}] + b_i)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot tanh(c_t)$$

$$z_t = \sigma(W_z [x_t, h_{t-1}] + b_z), \quad r_t = \sigma(W_r [x_t, h_{t-1}] + b_r)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

For beginners, the most important idea is not the formula alone. It is the connection between the formula and the data type: tabular data suit Dense layers, images suit convolutional layers, and ordered signals suit recurrent or attention-based layers.

### 6.6 Normalization and regularization layers

BatchNormalization standardizes intermediate activations and then learns a scale-and-shift correction. Dropout randomly suppresses some activations during training so the model does not rely too strongly on any one internal path [25], [26].

$$\hat{x} = (x - \mu) / sqrt(\sigma^2 + \varepsilon), \quad y = \gamma \hat{x} + \beta$$

$$x' = m \odot x, \quad m_i \sim Bernoulli(1 - p)$$

For beginners, the most important idea is not the formula alone. It is the connection between the formula and the data type: tabular data suit Dense layers, images suit convolutional layers, and ordered signals suit recurrent or attention-based layers.

### 6.7 Attention layers

Attention computes relevance scores between a query tensor and a set of value positions. These scores become normalized weights, and the final context vector becomes a weighted combination of the values. This lets the model focus selectively instead of compressing all sequence information into one fixed state [9], [18].

$$score = Q K^T$$

$$weights = softmax(score), \quad context = weights V$$

For beginners, the most important idea is not the formula alone. It is the connection between the formula and the data type: tabular data suit Dense layers, images suit convolutional layers, and ordered signals suit recurrent or attention-based layers.

**Section summary**

- The equations are not separate from the code. Each Keras layer is an implementation of one of these mathematical patterns.
- Convolution uses local weight sharing, recurrence carries state through time, and attention builds weighted focus over positions.
- A strong beginner habit is to ask both “What does this equation compute?” and “What data shape makes that computation meaningful?”

## 7 TensorFlow blocks and architectural patterns

A block is a small reusable arrangement of layers that appears often in practical networks. Thinking in blocks helps beginners move from isolated layers to real model design. In Keras, a block can be written as a function, a small Sequential object, or a custom layer class.

### 7.1 Convolutional block

A convolutional block groups layers such as Conv2D, BatchNormalization, ReLU, and MaxPooling2D into one reusable pattern. It is the standard beginner building block for image models.

**Mechanical Engineering perspective:** Use this pattern for crack patches, machined-surface defect patches, or drone images of structural surfaces.

```
import tensorflow as tf

def conv_block(filters):
    return tf.keras.Sequential(
        [
            tf.keras.layers.Conv2D(filters, 3, padding="same"),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.ReLU(),
            tf.keras.layers.MaxPooling2D(pool_size=2),
        ],
        name=f"conv_block_{filters}",
    )
```

Input and output meaning: the block receives one tensor and returns a transformed tensor that can be passed to the next stage of the model. The exact shape change depends on the block design, especially whether pooling, concatenation, or upsampling is used.

### 7.2 Residual block

A residual block adds a shortcut path so the block learns a correction to the input rather than a completely new mapping. This idea is central to residual networks [16].

**Mechanical Engineering perspective:** Residual connections are useful when building deeper inspection models without losing early texture information.

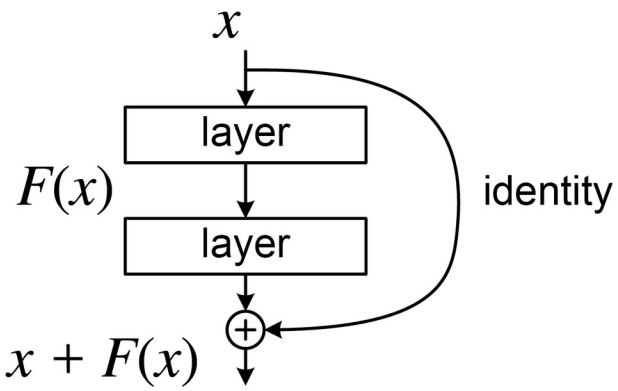


***Figure 7. Residual block with a shortcut path added to the transformed path. The Add layer preserves shape and helps information and gradients flow through deeper networks. Source: LunarLullaby via Wikimedia Commons, CC BY-SA 4.0 [30]***

Figure 7 shows the essential residual idea: the block output is the transformed path plus the shortcut path. This design underlies ResNet-style image models [16], [30].

```
import tensorflow as tf

def residual_block(x, filters):
    shortcut = x

    x = tf.keras.layers.Conv2D(filters, 3, padding="same")(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU()(x)

    x = tf.keras.layers.Conv2D(filters, 3, padding="same")(x)
    x = tf.keras.layers.BatchNormalization()(x)

    if shortcut.shape[-1] != filters:
        shortcut = tf.keras.layers.Conv2D(filters, 1, padding="same")(shortcut)

    x = tf.keras.layers.Add()([x, shortcut])
    x = tf.keras.layers.ReLU()(x)
    return x
```

Input and output meaning: the block receives one tensor and returns a transformed tensor that can be passed to the next stage of the model. The exact shape change depends on the block design, especially whether pooling, concatenation, or upsampling is used.

### 7.3 Inception-style block

An Inception-style block processes the same input with different kernel sizes in parallel and then concatenates the results. This supports multi-scale reasoning [17].

**Mechanical Engineering perspective:** This is helpful when cracks or surface defects appear at different sizes.

```
import tensorflow as tf

def inception_style_block(x, filters):
    b1 = tf.keras.layers.Conv2D(filters, 1, padding="same", activation="relu")(x)
    b3 = tf.keras.layers.Conv2D(filters, 3, padding="same", activation="relu")(x)
    b5 = tf.keras.layers.Conv2D(filters, 5, padding="same", activation="relu")(x)

    bp = tf.keras.layers.MaxPooling2D(pool_size=3, strides=1, padding="same")(x)
    bp = tf.keras.layers.Conv2D(filters, 1, padding="same", activation="relu")(bp)

    return tf.keras.layers.Concatenate()([b1, b3, b5, bp])
```

Input and output meaning: the block receives one tensor and returns a transformed tensor that can be passed to the next stage of the model. The exact shape change depends on the block design, especially whether pooling, concatenation, or upsampling is used.

### 7.4 Dense-connectivity block

A dense-connectivity block feeds each new layer with the outputs of all earlier layers in the block. This is different from a Dense layer. Here, “dense” refers to connectivity between layers.

**Mechanical Engineering perspective:** The pattern can help preserve low-level crack or texture information in image tasks.

```
import tensorflow as tf

def dense_connectivity_block(x, growth_rate=16, n_layers=3):
    features = [x]

    for _ in range(n_layers):
        y = tf.keras.layers.Concatenate()(features)
        y = tf.keras.layers.BatchNormalization()(y)
        y = tf.keras.layers.ReLU()(y)
        y = tf.keras.layers.Conv2D(growth_rate, 3, padding="same")(y)
        features.append(y)

    return tf.keras.layers.Concatenate()(features)
```

Input and output meaning: the block receives one tensor and returns a transformed tensor that can be passed to the next stage of the model. The exact shape change depends on the block design, especially whether pooling, concatenation, or upsampling is used.

### 7.5 Bottleneck block

A bottleneck block first reduces channel count with a 1 x 1 convolution, performs a more expensive spatial convolution on the smaller representation, and then expands again if needed. It saves computation.

**Mechanical Engineering perspective:** Useful when high-resolution inspection images must be processed efficiently.

```
import tensorflow as tf

def bottleneck_block(x, reduced_filters=16, output_filters=32):
    shortcut = x

    x = tf.keras.layers.Conv2D(reduced_filters, 1, padding="same", activation="relu")(x)
    x = tf.keras.layers.Conv2D(reduced_filters, 3, padding="same", activation="relu")(x)
    x = tf.keras.layers.Conv2D(output_filters, 1, padding="same")(x)

    if shortcut.shape[-1] != output_filters:
        shortcut = tf.keras.layers.Conv2D(output_filters, 1, padding="same")(shortcut)

    x = tf.keras.layers.Add()([x, shortcut])
    return tf.keras.layers.ReLU()(x)
```

Input and output meaning: the block receives one tensor and returns a transformed tensor that can be passed to the next stage of the model. The exact shape change depends on the block design, especially whether pooling, concatenation, or upsampling is used.

### 7.6 LSTM or GRU block

A sequence block often stacks one recurrent layer that returns a full sequence and a second recurrent or Dense layer that summarizes it. Dropout is commonly inserted between them.

**Mechanical Engineering perspective:** This is a practical starting structure for sensor-based structural health monitoring, water-level forecasting, and production throughput sequence modeling.

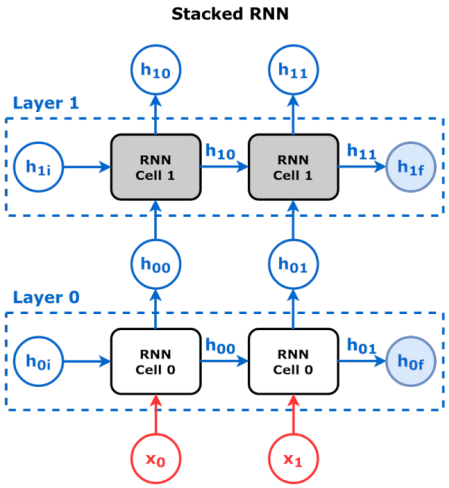


*Figure 8. A stacked recurrent network. Outputs from one recurrent layer become the inputs to another recurrent layer. Source: Daniel Voigt Godoy via Wikimedia Commons, CC BY 4.0 [35]*

Figure 8 helps students see that “stacking” means feeding the sequence representation learned by one recurrent layer into another recurrent layer above it [35].

```
import tensorflow as tf

def sequence_block(kind="lstm", units=32):
  recurrent = tf.keras.layers.LSTM if kind == "lstm" else tf.keras.layers.GRU

  return tf.keras.Sequential(
    [
      recurrent(units, return_sequences=True),
      tf.keras.layers.Dropout(0.2),
      recurrent(units),
    ],
    name=f"{kind}_block",
  )
```

Input and output meaning: the block receives one tensor and returns a transformed tensor that can be passed to the next stage of the model. The exact shape change depends on the block design, especially whether pooling, concatenation, or upsampling is used.

### 7.7 Upsampling block and transposed convolution block

Decoder-side image models often enlarge feature maps either with fixed upsampling followed by Conv2D or with a learned Conv2DTranspose layer.

**Mechanical Engineering perspective:** Useful for heat maps, segmentation-like damage localization, and image reconstruction tasks.

```
import tensorflow as tf

def upsample_block(x, filters):
  x = tf.keras.layers.UpSampling2D(size=2)(x)
  x = tf.keras.layers.Conv2D(filters, 3, padding="same", activation="relu")(x)
  return x

def transposed_conv_block(x, filters):
  return tf.keras.layers.Conv2DTranspose(
```

```
    filters, 3, strides=2, padding="same", activation="relu"
)(x)
```

Input and output meaning: the block receives one tensor and returns a transformed tensor that can be passed to the next stage of the model. The exact shape change depends on the block design, especially whether pooling, concatenation, or upsampling is used.

### 7.8 Attention block

An attention block lets a sequence or feature map focus on its most relevant positions before the next layer makes a decision.

**Mechanical Engineering perspective:** For sequence data, attention can highlight the most informative past sensor readings. For image-related reasoning, it can emphasize locations associated with likely damage.

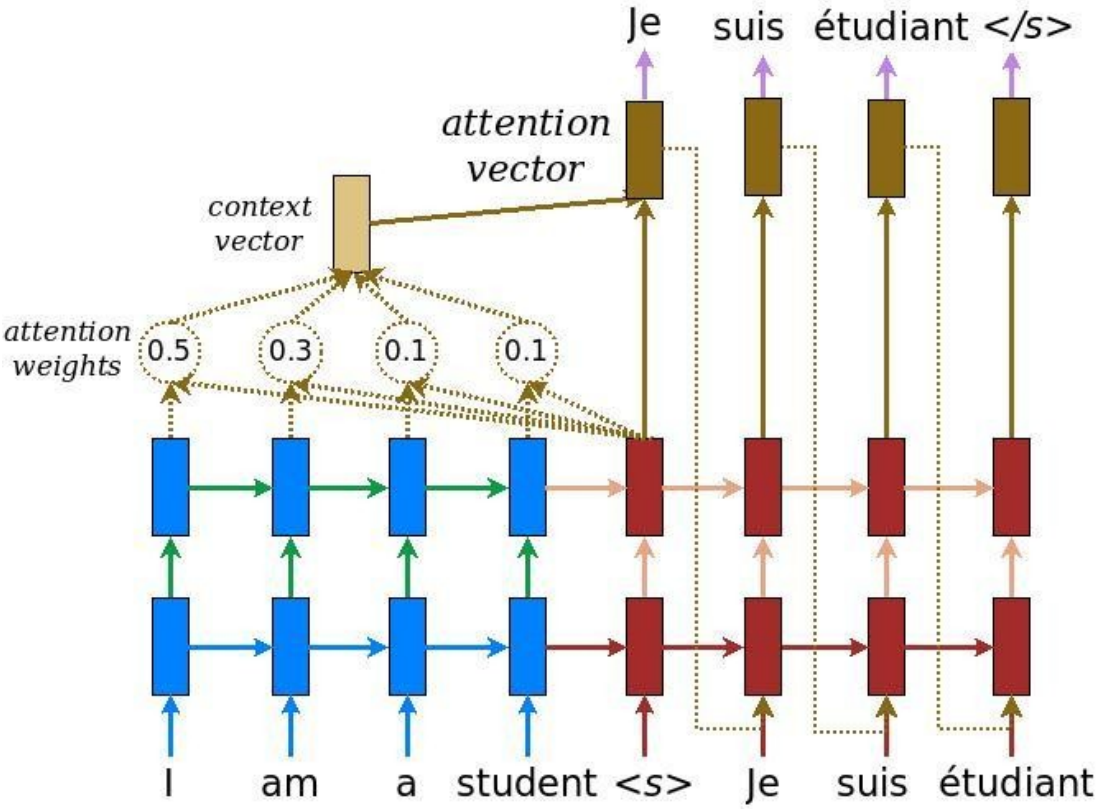


***Figure 9. Attention in sequence-to-sequence processing. Attention weights determine which earlier positions influence the current output most strongly. Source: TensorFlow Tutorials [34]***

Figure 9 makes attention material: instead of forcing all earlier information into one fixed state, the model can focus on different earlier positions for different outputs [34].

```
import tensorflow as tf

def attention_block(x):
  attended = tf.keras.layers.Attention()([x, x])
  x = tf.keras.layers.Add()([x, attended])
  x = tf.keras.layers.LayerNormalization()(x)
  return x
```

Input and output meaning: the block receives one tensor and returns a transformed tensor that can be passed to the next stage of the model. The exact shape change depends on the block design, especially whether pooling, concatenation, or upsampling is used.

### 7.9 Transformer-style block

A Transformer-style block uses self-attention, residual Add operations, normalization, and a feed-forward network. It is a flexible sequence block for long-range interactions [18].

**Mechanical Engineering perspective:** This style is increasingly useful for long sensor histories, production throughput data, and hydrologic sequences where important information may be far apart in time.

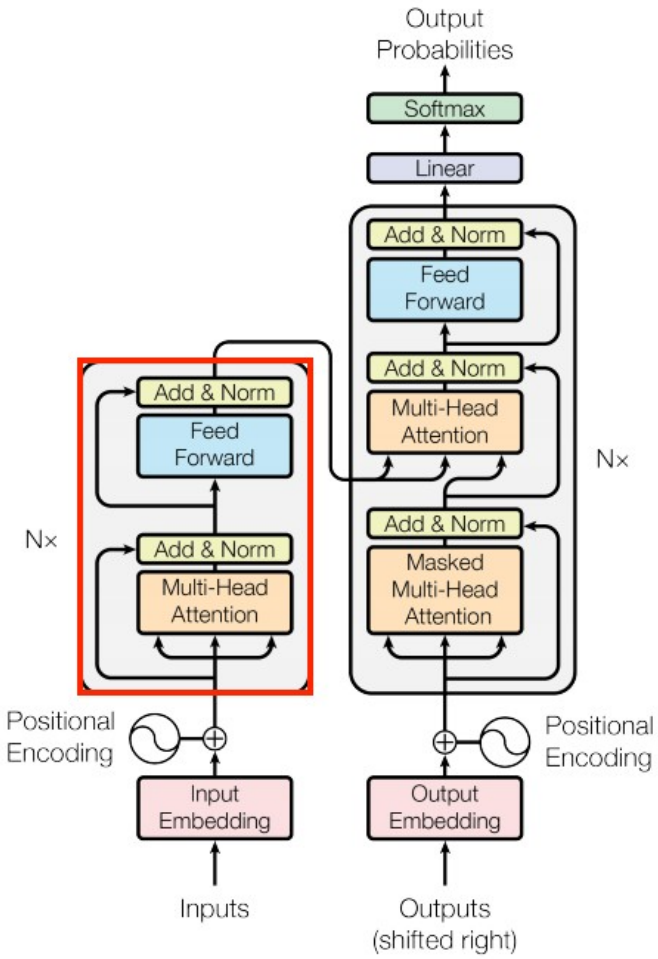


***Figure 10. Transformer encoder and decoder structure. For beginners, focus on the encoder-side pattern highlighted by self-attention, Add, normalization, and a feed-forward network. Source: TensorFlow Tutorials [36]***

Figure 10 is helpful because it shows the repeated structure of a Transformer-style sequence block. Even when beginners do not yet build a full transformer, they can learn the idea of self-attention plus residual and feed-forward steps [36].

```
import tensorflow as tf

def transformer_style_block(x, num_heads=4, key_dim=16, ff_dim=64, dropout=0.1):
    attention_output = tf.keras.layers.MultiHeadAttention(
```

```
    num_heads=num_heads,
    key_dim=key_dim,
    dropout=dropout,
)(x, x)

x = tf.keras.layers.Add()([x, attention_output])
x = tf.keras.layers.LayerNormalization()(x)

ff = tf.keras.layers.Dense(ff_dim, activation="relu")(x)
ff = tf.keras.layers.Dense(int(x.shape[-1]))(ff)

x = tf.keras.layers.Add()([x, ff])
x = tf.keras.layers.LayerNormalization()(x)
return x
```

Input and output meaning: the block receives one tensor and returns a transformed tensor that can be passed to the next stage of the model. The exact shape change depends on the block design, especially whether pooling, concatenation, or upsampling is used.

### Section summary

- Blocks are reusable layer patterns that make model design more systematic.
- Residual, dense-connectivity, and Inception-style blocks change how information moves between layers, not just which layers are used.
- Sequence blocks, attention blocks, and Transformer-style blocks are especially relevant for structural monitoring and forecasting tasks.

## 8 Models in TensorFlow

A model is the complete learnable mapping from input tensors to output tensors. In Keras, a model is more than a list of layers. It is also the object that can be compiled, trained, evaluated, saved, and used for inference [2], [5], [7].

### 8.1 Sequential models

A Sequential model is the simplest style. It works well when the network is a straight stack in which each layer has one input and one output, and information flows only forward through the stack [3].

```
import tensorflow as tf

model = tf.keras.Sequential(
    [
        tf.keras.Input(shape=(8,), name="mix_features"),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(1, name="strength_mpa"),
    ],
    name="sequential_regressor",
)

model.summary()
```

This is a good starting point for material strength prediction, material-property regression, or any other single-input tabular problem.

### 8.2 Functional API models

The Functional API is more flexible. It should be used when a model has branches, merges, several inputs, several outputs, shared layers, or skip connections [4].

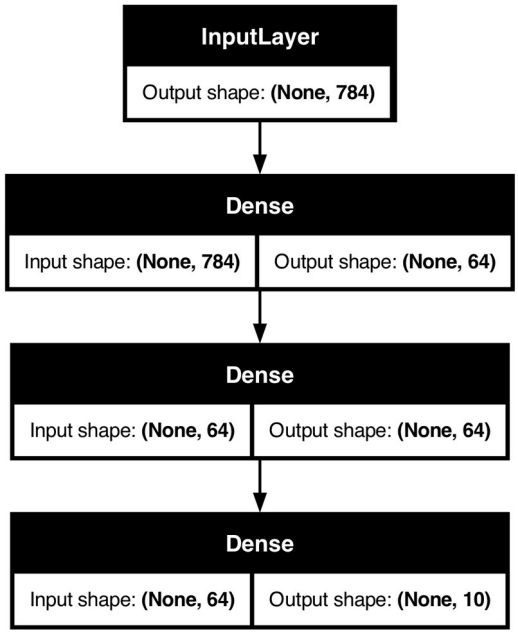


***Figure 11. Example of a non-linear model graph in the Keras Functional API. The diagram shows that model design can branch and merge rather than staying as a simple layer stack. Source: Keras Guides [4]***

Figure 11 is particularly useful for beginners because it visualizes a model as a graph of tensor flow rather than as a plain list. That graph view becomes essential when using Add, Concatenate, multiple inputs, or multiple outputs [4].

```
import tensorflow as tf

image_in = tf.keras.Input(shape=(128, 128, 3), name="surface_image")
meta_in = tf.keras.Input(shape=(4,), name="site_metadata")

# Image branch
x1 = tf.keras.layers.Conv2D(16, 3, activation="relu")(image_in)
x1 = tf.keras.layers.MaxPooling2D()(x1)
x1 = tf.keras.layers.Conv2D(32, 3, activation="relu")(x1)
x1 = tf.keras.layers.GlobalAveragePooling2D()(x1)

# Metadata branch
x2 = tf.keras.layers.Dense(16, activation="relu")(meta_in)

# Merge the branches
x = tf.keras.layers.Concatenate()([x1, x2])
x = tf.keras.layers.Dense(32, activation="relu")(x)
out = tf.keras.layers.Dense(4, activation="softmax", name="distress_class")(x)

model = tf.keras.Model(
    inputs=[image_in, meta_in],
    outputs=out,
    name="machined surface_fusion_model",
)

model.summary()
```

This pattern is valuable when an engineering decision depends on more than one data source. For example, an image branch may learn visual distress features while a metadata branch learns how age, production throughput, and climate context affect interpretation.

### 8.3 When should a beginner use each approach?

#### Sequential versus Functional API

| Question | Sequential | Functional API |
|---|---|---|
| Best for | Straight layer stacks. | Branched, merged, shared, multi-input, or skip-connected graphs. |
| Difficulty | Easier for the first model. | Slightly more abstract but more flexible. |
| Mechanical Engineering starter use | Material strength regression. | Image plus metadata fusion for distress classification. |
| Can express residual or multi-branch design? | Not naturally. | Yes. |

A practical beginner strategy is this: start with Sequential when the network really is a straight stack; move to the Functional API as soon as the problem naturally needs two inputs, two outputs, branching, merging, or skip connections.

### Section summary

- Use Sequential for a plain stack of layers.
- Use the Functional API for realistic engineering graphs with branching, merging, or multiple inputs.
- A model is the full trainable object, while a layer is one transformation inside that object.

## 9 Custom layers in TensorFlow

Built-in layers cover many needs, but not every need. A custom layer is appropriate when the computation you want is specific to your problem, when you want a reusable engineering transformation, or when you want a cleaner abstraction than copying the same low-level code repeatedly. Keras provides a beginner-friendly subclassing pattern for this purpose [6].

### 9.1 Why a custom layer might be needed

Examples include learning a special way to combine sensor channels, building an attention-like summary tailored to engineering sequences, or encapsulating a repeated block that should behave like one reusable layer object inside larger models.

### 9.2 Subclassing in simple Python terms

Subclassing means creating a new class that inherits behavior from an existing class. When you inherit from tf.keras.layers.Layer, your new class automatically gains the bookkeeping that Keras uses to track weights, trainable variables, and nested layers.

**The three most important methods in a custom layer**

| Method | What it is for | Beginner memory aid |
|---|---|---|
| __init__ | Store configuration that is known when the object is created. | “What settings does this layer have?” |
| build | Create weights once the input shape is known. | “What variables must exist for this input size?” |
| call | Define the forward computation on the input tensor. | “What math happens when the layer is used?” |

### 9.3 Example 1: a trainable feature-affine layer

This first example is intentionally simple. It learns one scale and one offset for each input feature. This can be interpreted as a learnable feature calibration step for tabular or sensor inputs.

```
import tensorflow as tf

class FeatureAffine(tf.keras.layers.Layer):
    def __init__(self, **kwargs):
        # Call the parent constructor so Keras can set up the layer.
        super().__init__(**kwargs)

    def build(self, input_shape):
        # input_shape[-1] is the number of features in each sample.
        n_features = int(input_shape[-1])

        # One trainable scale per feature.
        self.scale = self.add_weight(
            name="scale",
            shape=(n_features,),
            initializer="ones",
            trainable=True,
        )
```

```
        # One trainable offset per feature.
        self.offset = self.add_weight(
            name="offset",
            shape=(n_features,),
            initializer="zeros",
            trainable=True,
        )

    def call(self, inputs):
        # Apply featurewise affine transformation.
        return inputs * self.scale + self.offset

x = tf.random.normal((4, 8))
layer = FeatureAffine()
y = layer(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
print("Trainable weights:", [w.shape for w in layer.trainable_weights])
```

| | |
|---|---|
| **What __init__ does** | It stores configuration known at creation time. In this simple example, there is no extra configuration to store. |
| **What build does** | It waits until the layer sees an input shape, then creates one trainable scale and one trainable offset per feature. |
| **What call does** | It performs the actual forward computation: output = input × scale + offset. |
| **Mechanical Engineering interpretation** | A feature-affine layer can learn that some sensor channels or material variables should be amplified, attenuated, or shifted before the next layer uses them. |

### 9.4 Example 2: attention-style pooling over a sensor sequence

The next custom layer is still beginner-friendly, but slightly richer. It learns which time steps deserve more weight when summarizing a sequence into one vector.

```
import tensorflow as tf

class SensorAttentionPooling(tf.keras.layers.Layer):
    def __init__(self, hidden_units=16, **kwargs):
        super().__init__(**kwargs)
        # These are built-in layers used inside the custom layer.
        self.score_hidden = tf.keras.layers.Dense(hidden_units, activation="tanh")
        self.score_out = tf.keras.layers.Dense(1)

    def call(self, inputs):
        # inputs shape: (batch, time_steps, features)
        scores = self.score_hidden(inputs)
        scores = self.score_out(scores)          # (batch, time_steps, 1)
        weights = tf.nn.softmax(scores, axis=1)   # attention over time
        context = tf.reduce_sum(inputs * weights, axis=1)
```

```
        return context

x = tf.random.normal((2, 60, 8))
pool = SensorAttentionPooling(hidden_units=12)
y = pool(x)

print("Input shape:", x.shape)
print("Output shape:", y.shape)
```

This layer is useful when a whole sensor window should become one summary vector, but not all time steps should be treated equally. In a structural monitoring setting, the layer may learn to pay more attention to time steps associated with unusual vibration or response patterns.

### 9.5 Putting a custom layer inside a model

```
import tensorflow as tf

sequence_in = tf.keras.Input(shape=(60, 8), name="sensor_window")
x = SensorAttentionPooling(hidden_units=12)(sequence_in)
out = tf.keras.layers.Dense(1, name="damage_score")(x)

model = tf.keras.Model(sequence_in, out, name="sensor_attention_model")
model.summary()
```

This example shows the main idea clearly: once a custom layer is written, it can be used inside a normal Keras model just like any built-in layer.

### 9.6 Good beginner guidelines for custom layer design

- Start with the smallest useful idea. A custom layer should solve one clear problem.
- Create weights in build when their size depends on the incoming shape.
- Keep the call method focused on forward computation only.
- Print shapes when debugging the first version of a new layer.
- Only make a custom layer when built-in layers do not already express the idea clearly enough.

**Section summary**

- Custom layers let you package new engineering computations as reusable Keras objects.
- The beginner pattern is __init__ for configuration, build for weights, and call for forward computation.
- Once written, a custom layer can be inserted into a normal model exactly like a built-in layer.

### 10 Mechanical Engineering mini-cases and recommended starting points

The table below summarizes practical starting architectures for common Mechanical Engineering problem types. These are not the only valid choices, but they are reasonable first baselines for student work. Earlier sections already explained the layers that appear in these designs.

| Problem | Typical input | Recommended beginner baseline | Output |
|---|---|---|---|
| Material strength prediction | (batch, 8) tabular features | Sequential model with Dense -> Dense -> Dense(1) | One scalar strength value |
| Machined-surface defect image classification | (batch, H, W, 3) image | Conv block -> Conv block -> GlobalAveragePooling2D -> Dense -> Softmax | Class probabilities |
| Crack or damage patch classification | (batch, H, W, 1 or 3) image | Small CNN or residual CNN | Damage / no-damage or several classes |
| Structural health monitoring window classification | (batch, T, F) sequence | GRU or LSTM block -> Dense | Class or score |
| Water-level or coolant flow forecasting | (batch, T, F) sequence | LSTM or attention-based sequence model | Future scalar or future sequence |
| Vehicle systems sequences | (batch, T, F) sequence | GRU, LSTM, or Transformer-style block | Forecast or class |
| Material or material classification | (batch, F) tabular features | Dense classifier with Softmax output | Class probabilities |
| Monitoring anomaly detection idea | Tabular, image, or sequence data | Encoder-decoder or reconstruction-style model using Conv2D / Conv2DTranspose or sequence layers | Anomaly score or reconstruction error |

#### 10.1 Material strength prediction

This is one of the cleanest first TensorFlow projects because the data are tabular and the output is a single scalar. Start with Dense layers and keep the baseline simple before trying more advanced ideas [19], [20].

#### 10.2 Machined-surface defect image classification

The basic pattern is convolutional feature extraction followed by a compact classifier. If site metadata such as workcell type, production line age, or production throughput level matter, add a second metadata branch with the Functional API and merge with Concatenate [22].

### 10.3 Crack detection and damage-region reasoning

Convolutional blocks and residual blocks are natural building elements. For location-sensitive outputs, use upsampling or transposed convolution in a decoder-style design. For patch-level classification, a simpler CNN is often enough [21].

### 10.4 Structural health monitoring from sensor time series

Sensor data are ordered, so sequence-aware layers matter. Start with GRU or LSTM. If a long time window contains only a few crucial events, add an attention mechanism or a custom attention-style pooling layer [14], [15], [23].

### 10.5 Flow-rate-coolant flow or water-level forecasting

Respect causality in forecasting problems. Use only past information to predict the future. Bidirectional recurrence is usually less appropriate for strict causal forecasting than causal recurrent or attention-based sequence models.

### 10.6 Vehicle systems sequence data

These problems may use tabular, image, graph, or sequence inputs. For a beginner baseline, sequence windows with GRU, LSTM, or a small Transformer-style block are a practical starting point.

### 10.7 Material or material classification

If the inputs are ordinary numerical measurements, begin with Dense layers. If there are integer-coded categorical descriptors such as material class IDs or production line class IDs, Embedding may be useful.

### 10.8 General project advice for Mechanical Engineering data

- Normalize or standardize numerical features before training.
- Split train, validation, and test data carefully so that leakage across structures, sites, or time windows does not inflate results.
- Start with the simplest credible baseline and add complexity only when the baseline is understood.
- In image tasks, check class imbalance and label quality. In sensor tasks, check missing data and time alignment.
- Interpret predictions in engineering units and domain language, not only in machine-learning metrics.

**Section summary**

- Choose the first architecture from the data type: tabular, image, or sequence.
- Keep the first baseline simple and domain-aware.
- Mechanical Engineering success depends on good splits, good labels, and physically meaningful evaluation, not only on deeper networks.

## 11 Practical debugging and beginner mistakes

Beginners often think a TensorFlow problem is “advanced” when the real issue is a mismatch between shapes, losses, data types, or training versus inference behavior. The checklist below is designed to reduce that frustration.

### Common beginner issues

| Symptom | Likely cause | How to inspect or fix |
|---|---|---|
| Shape mismatch error | Input tensor shape does not match what a layer expects. | Print x.shape before and after major layers. |
| Add layer error | The tensors sent to Add do not have the same shape. | Compare both shapes and project one branch if needed. |
| Unexpected output size after a recurrent stack | Forgot return_sequences=True on an earlier recurrent layer. | Check whether the next layer expects a sequence or only a final summary vector. |
| Embedding error | Input values are float measurements instead of integer indices. | Use integer-coded categories only. |
| Poor regression results | Output layer or loss does not match the task. | For scalar regression, use one output unit and a regression loss such as MSE or MAE. |
| Dropout or BatchNormalization behaves strangely at prediction time | Training and inference modes differ. | Remember that these layers act differently in training and inference. |

### 11.1 A minimal inspection toolkit

```
import tensorflow as tf

print("Tensor shape:", x.shape)
print("Dynamic shape:", tf.shape(x))
print("Data type:", x.dtype)

print(model.summary())

for w in model.weights:
    print(w.name, w.shape, "trainable=" + str(w.trainable))
```

If something looks wrong, reduce the model to the smallest failing example. One layer with one random input tensor is often enough to reveal the issue.

### 11.2 Choosing the right output layer

| | |
|---|---|
| **Scalar regression** | Use one output unit, often without Softmax. Examples include strength, deflection, or water-level prediction. |

| | |
|---|---|
| **Binary classification** | Use one output unit with a binary-style interpretation or two-class Softmax if designed consistently. |
| **Multi-class classification** | Use one output unit per class and a Softmax output. |
| **Sequence output** | Use return_sequences=True in the final sequence-producing stage if one output is needed at each time step. |

### 11.3 A final beginner rule

Whenever a layer feels mysterious, ask four questions: What shape goes in? What shape comes out? Does it learn parameters? Why is that useful for this data type? If you can answer those four questions, you usually understand the layer well enough to use it correctly.

#### Section summary

- Most TensorFlow beginner problems are shape, loss, or data-type problems, not mysterious optimization problems.
- Print shapes and weights often.
- Always match the output layer and loss function to the engineering task.

## 12 Conclusion

TensorFlow becomes much less intimidating once the subject is organized around a small set of questions: What kind of data do I have? What tensor shape represents that data? Which layers are natural for that shape? How do those layers combine into a model? And which parts of the computation should be learned?

For Mechanical Engineering students, the most important lesson is that the software should serve the engineering question. A model for material strength is not built the same way as a model for crack images or a model for sensor sequences. TensorFlow is valuable because it gives one consistent framework for all of these problem types while still letting the engineer choose layers and blocks appropriate to the data and the physical task.

The recommended next step after reading this tutorial is to implement one complete mini project in each of three data types: tabular regression, image classification, and sequence forecasting or monitoring. That practice will make the abstract vocabulary of tensors, layers, and models feel material and usable.

## References

[1] TensorFlow Team, 'What's new in TensorFlow 2.20,' TensorFlow Blog, Aug. 19, 2025. [Online]. Available: https://blog.tensorflow.org/2025/08/whats-new-in-tensorflow-2-20.html. Accessed: Mar. 28, 2026.

[2] TensorFlow, 'Keras: The high-level API for TensorFlow,' TensorFlow Guide. [Online]. Available: https://www.tensorflow.org/guide/keras. Accessed: Mar. 28, 2026.

[3] TensorFlow, 'The Sequential model,' TensorFlow Guide. [Online]. Available: https://www.tensorflow.org/guide/keras/sequential_model. Accessed: Mar. 28, 2026.

[4] Keras, 'Functional API,' Keras Guides. [Online]. Available: https://keras.io/guides/functional_api/. Accessed: Mar. 28, 2026.

[5] TensorFlow, 'Training & evaluation with the built-in methods,' TensorFlow Guide. [Online]. Available: https://www.tensorflow.org/guide/keras/training_with_built_in_methods. Accessed: Mar. 28, 2026.

[6] Keras, 'Making new layers and models via subclassing,' Keras Guides. [Online]. Available: https://keras.io/guides/making_new_layers_and_models_via_subclassing/. Accessed: Mar. 28, 2026.

[7] Keras, 'Layers API,' Keras API. [Online]. Available: https://keras.io/api/layers/. Accessed: Mar. 28, 2026.

[8] Keras, 'Conv2D layer,' Keras API. [Online]. Available: https://keras.io/api/layers/convolution_layers/convolution2d/. Accessed: Mar. 28, 2026.

[9] TensorFlow, 'tf.keras.layers.Attention,' TensorFlow API. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Attention. Accessed: Mar. 28, 2026.

[10] TensorFlow, 'tf.shape,' TensorFlow API. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/shape. Accessed: Mar. 28, 2026.

[11] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.

[12] F. Chollet, Deep Learning with Python, 2nd ed. Shelter Island, NY: Manning, 2021.

[13] S. Haykin, Neural Networks and Learning Machines, 3rd ed. New York, NY: Pearson, 2009.

[14] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[15] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," arXiv:1406.1078, 2014.

[16] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2016, pp. 770-778.

[17] C. Szegedy et al., "Going deeper with convolutions," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2015, pp. 1-9.

[18] A. Vaswani et al., "Attention is all you need," in Proc. Advances in Neural Information Processing Systems, 2017, pp. 5998-6008.

[19] I.-C. Yeh, "Modeling of strength of high-performance material using artificial neural networks," Cement and Material Research, vol. 28, no. 12, pp. 1797-1808, 1998.

[20] I.-C. Yeh, 'Mechanical Material Strength' [Dataset], UCI Machine Learning Repository. [Online]. Available: https://archive.ics.uci.edu/dataset/165/concrete+compressive+strength. Accessed: Mar. 28, 2026.

[21] Y.-J. Cha, W. Choi, and O. Büyüköztürk, "Deep learning-based crack damage detection using convolutional neural networks," Computer-Aided Mechanical and Mechanical systems Engineering, vol. 32, no. 5, pp. 361-378, 2017.

[22] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiyama, and H. Omata, "Production line damage detection and classification using deep neural networks with smartphone images," Computer-Aided Mechanical and Mechanical systems Engineering, vol. 33, no. 12, pp. 1127-1141, 2018.

[23] O. Abdeljaber, O. Avci, S. Kiranyaz, M. Gabbouj, and D. J. Inman, "1D CNNs for structural damage detection: verification on a structural health monitoring benchmark data," Neurocomputing, vol. 275, pp. 1308-1317, 2018.

[24] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in Proc. 14th Int. Conf. Artificial Intelligence and Statistics, 2011, pp. 315-323.

[25] S. Ioffe and C. Szegedy, "Batch normalization: accelerating deep network training by reducing internal covariate shift," in Proc. Int. Conf. Machine Learning, 2015, pp. 448-456.

[26] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929-1958, 2014.

[27] Wiso, 'Neural network example,' Wikimedia Commons, public domain. [Online]. Available: https://commons.wikimedia.org/wiki/File:Neural_network_example.svg. Accessed: Mar. 28, 2026.

[28] Renanar2, 'Convolutional Layers of a Convolutional Neural Network,' Wikimedia Commons, CC BY-SA 4.0. [Online]. Available: https://commons.wikimedia.org/wiki/File:Convolutional_Layers_of_a_Convolutional_Neural_Network.svg. Accessed: Mar. 28, 2026.

[29] D. Voigt Godoy, 'Convolutional neural network, maxpooling,' Wikimedia Commons, CC BY 4.0. [Online]. Available: https://commons.wikimedia.org/wiki/File:Convolutional_neural_network,_maxpooling.png. Accessed: Mar. 28, 2026.

[30] LunarLullaby, 'ResBlock,' Wikimedia Commons, CC BY-SA 4.0. [Online]. Available: https://commons.wikimedia.org/wiki/File:ResBlock.png. Accessed: Mar. 28, 2026.

[31] See-Ming Lee, 'NN LSTM-Cell v2,' Wikimedia Commons, CC0 1.0. [Online]. Available: https://commons.wikimedia.org/wiki/File:NN_LSTM-Cell_v2.svg. Accessed: Mar. 28, 2026.

[32] Wikimedia Commons contributors, 'Gated Recurrent Unit 3,' Wikimedia Commons. [Online]. Available: https://commons.wikimedia.org/wiki/File:Gated_Recurrent_Unit_3.svg. Accessed: Mar. 28, 2026.

[33] TensorFlow, 'Text classification with an RNN,' TensorFlow Tutorials. [Online]. Available: https://www.tensorflow.org/text/tutorials/text_classification_rnn. Accessed: Mar. 28, 2026.

[34] TensorFlow, 'Neural machine translation with attention,' TensorFlow Tutorials. [Online]. Available: https://www.tensorflow.org/text/tutorials/nmt_with_attention. Accessed: Mar. 28, 2026.

[35] D. Voigt Godoy, 'Stacked RNN,' Wikimedia Commons, CC BY 4.0. [Online]. Available: https://commons.wikimedia.org/wiki/File:Stacked_RNN.png. Accessed: Mar. 28, 2026.

[36] TensorFlow, 'Transformer model for language understanding,' TensorFlow Tutorials. [Online]. Available: https://www.tensorflow.org/text/tutorials/transformer. Accessed: Mar. 28, 2026.